AI-Powered Personalized Mock Interview and Adaptive Candidate Evaluation System

(Callback Rehearsal – InterviewAI)

A

Project Report submitted in partial fulfillment of the requirement

for the award of degree of

Master of Technology

in Artificial Intelligence and Data Science

Session: 2025-26

Submitted by

Vishal Rathod

Roll Number: [Roll Number]

Supervised By

[Supervisor Name], [Designation], SDSF, DAVV

School of Data Science and Forecasting

Devi Ahilya Vishwavidyalaya,

Indore, MP

Certificate of Originality

I, Vishal Rathod, hereby declare that this report, titled “AI-Powered Personalized Mock Interview and Adaptive Candidate Evaluation System”, is based on original work conducted by me under the supervision of [Supervisor Name], [Designation]. I have not plagiarized any material and have cited all sources used in this work.

All publications arising from this work are duly acknowledged, and I have obtained all necessary permissions from the copyright holders for any included material. I also confirm that this report has not been previously submitted for any other degree or diploma at this or any other institution.

I understand that submitting any false or misleading information in this subject will be considered a violation of academic integrity and may result in failing the course. I acknowledge my responsibility to uphold academic honesty in all my work.

Sign with date
Vishal Rathod
[Roll Number]

Certificate by Supervisor

I, [Supervisor Name], confirm that Vishal Rathod completed the report titled “AI-Powered Personalized Mock Interview and Adaptive Candidate Evaluation System” as part of his studies in the Master of Technology (Integrated) in Artificial Intelligence and Data Science at the School of Data Science and Forecasting, DAVV, under my supervision, and the submitted work is his original work.

Throughout the project, I provided guidance on research methods, analysis, and writing, and reviewed the final report before its submission.

Sign with date
[Supervisor Name]

Certificate of Completion

Certified that Vishal Rathod, with roll number [Roll Number], a student at the School of Data Science and Forecasting at DAVV, successfully completed the project titled “AI-Powered Personalized Mock Interview and Adaptive Candidate Evaluation System” from [Start Date] to [End Date].

I wish him all the best in his future endeavors.

Sign with date
[Head of Department / Authority Name]
[Designation]

Executive Summary

Area of work. The project lies at the intersection of Generative AI, Natural Language Processing, Retrieval-Augmented Generation (RAG), multi-agent systems and speech processing, applied to the problem of technical interview preparation.

Description of work. Callback Rehearsal – InterviewAI is a web application that turns a candidate's uploaded resume into a personalised, multi-round mock interview. The resume is parsed, converted into a structured profile and indexed as section-aware chunks in a FAISS vector store. A LangGraph state graph orchestrates three specialised interview agents (DSA coding, Technical and HR/behavioural), an Evaluation agent and a Report agent. Every question is generated with retrieval of the candidate's own resume chunks and carries source citations; every answer is scored against a standard rubric (technical correctness 40%, conceptual understanding 25%, answer completeness 20%, communication clarity 15%) and the score drives an adaptive difficulty policy that makes the next question easier, equal or harder. Interviews run in voice (Deepgram speech-to-text and ElevenLabs text-to-speech over WebSockets) or text, and conclude with analytics dashboards and a downloadable PDF report. The work started from an existing prototype; a full audit identified that its LangGraph graph was never executed, that retrieval was not used during interviews, that difficulty was fixed and that failed AI calls produced fabricated scores. These were replaced with an executed LangGraph turn graph, a real RAG pipeline, an adaptive engine, question-level persistence and honest provenance flags, together with a redesigned React interface.

Workplace / institution. The project was carried out as the M.Tech (Integrated) Minor Project at the School of Data Science and Forecasting, Devi Ahilya Vishwavidyalaya, Indore, using open-source frameworks (FastAPI, LangGraph, FAISS, React) and hosted large-language-model APIs (Groq and Google Gemini).

Objective and relevance. The objective is to give candidates realistic, personalised interview practice with measurable, explainable feedback, and to study three research questions: whether resume-aware retrieval makes questions more relevant, whether adaptive difficulty places candidates at an appropriate level, and how reliably an LLM-based evaluator scores answers. Experiments reported here show section-aware chunking raising retrieval Precision@1 from 0.33 to 0.94 on a labelled query set, and the adaptive policy keeping 71% of questions at the candidate's ability level against 29% for a fixed policy in simulation. The system is verified by 57 automated tests and a live end-to-end run with the production LLM. The result is a complete, documented, reproducible platform that is relevant to students preparing for placements and to research on LLM-based assessment.

Index

S.No.	Points	Page Number
1	Domain of Project	7
2	Utility of Work Done	8
3	Objectives	9
4	Technical Description	10
5	Methodology	17
6	Outcomes	19
7	Knowledge / Skill Improvement	23
8	Applications	24
9	Conclusions	25
10	Limitations	26
11	Future Directions	27
12	References	28
	Annexures	30

1. Domain of Project

The project belongs to the domain of applied Artificial Intelligence, specifically Generative AI and Natural Language Processing. It combines several sub-fields that are currently central to industry practice:

- **Large Language Models (LLMs).** Question generation, answer evaluation and report narration are performed by hosted LLMs (Groq-served open-weight models, with Google Gemini as an alternative provider).
- **Retrieval-Augmented Generation (RAG).** The candidate's resume is chunked, embedded and stored in a FAISS vector index; the most relevant chunks are retrieved for every question so that the interviewer talks about the candidate's real projects and experience.
- **Multi-agent systems.** A LangGraph state graph coordinates specialised agents (DSA, Technical, HR, Evaluation, Report) through a single orchestrator with conditional routing and shared state.
- **Adaptive assessment.** A configurable difficulty policy adjusts question difficulty from rubric scores, borrowing the idea of computerised adaptive testing.
- **Speech processing.** Real-time speech-to-text (Deepgram) and text-to-speech (ElevenLabs, with Google TTS fallback) over WebSockets make the interview conversational.
- **Full-stack engineering.** FastAPI, SQLAlchemy, PostgreSQL/SQLite, React 19, Vite and Tailwind CSS provide the production platform.

The application area is recruitment and career preparation: realistic, personalised interview practice for students and job seekers, together with an experimental test-bed for studying LLM-based question generation and evaluation.

2. Utility of Work Done

Technical interviews are high-stakes, and realistic practice is scarce. Existing resources are either static question banks or generic chatbots that ignore the candidate's background. They do not ask about the candidate's own projects, do not adapt to the candidate's level, and do not separate technical correctness from fluent delivery. The work done in this project addresses these gaps:

- **Personalisation.** Questions are grounded in the uploaded resume through retrieval and carry citations such as "projects#3" so the candidate can see why a question was asked.
- **Adaptivity.** Each answer's rubric score moves the next question easier, equal or harder, so strong candidates are stretched and weaker candidates are tested on fundamentals rather than being lost.
- **Explainable feedback.** Every answer receives four rubric scores with quoted evidence, gaps, ideal-answer points and learning topics; every round and session is summarised in a PDF report and in analytics dashboards.
- **Realism.** Voice interviews with explicit conversational states, three distinct rounds (coding in an editor, technical, behavioural) and interviewer personas simulate an actual recruitment cycle.
- **Honesty of results.** When an AI service is unavailable, the system does not invent scores; it falls back to a curated question bank and clearly flagged heuristic estimates. This was a deliberate correction of the prototype, which previously reported fixed "85/100 – recommend hire" style results on failure.
- **Reproducible research artefacts.** Experiment scripts, a human-rating template, test suites and ten documentation files allow the work to be verified and extended.

For the institution and future students the platform provides a free, self-hosted alternative to commercial mock-interview services, and a working reference implementation of RAG and LangGraph orchestration that can be reused in other projects.

3. Objectives

The project set out the following objectives, all of which were implemented and verified:

1. Extract a structured candidate profile (skills, programming languages, projects, experience, education, certifications) from an uploaded PDF or DOCX resume and present it in a resume-analysis dashboard without inventing information.
2. Index the resume as section-aware chunks with metadata in FAISS and retrieve relevant chunks during the interview, returning source references for every question.
3. Generate interview questions that are aware of the resume, the target job role and an optional job description, across twelve technical categories plus resume-project and behavioural questions, with follow-ups and duplicate detection.
4. Orchestrate DSA, Technical, HR, Evaluation and Report agents through a LangGraph state graph that is actually executed on every turn, with the orchestrator as the only controller.
5. Evaluate every answer with a standard rubric (40/25/20/15) producing structured JSON, and adapt the difficulty of the next question using configurable thresholds (score ≤ 4 decrease, 5–7 maintain, ≥ 8 increase).
6. Support real-time voice interviews end to end (microphone → Deepgram → agent → ElevenLabs → browser) with explicit interview states, duplicate-event prevention and a text fallback.
7. Persist every question, answer, score and difficulty decision in PostgreSQL (SQLite in development), and provide analytics dashboards, interview history and downloadable PDF reports built only from persisted data.
8. Redesign the frontend as a consistent, responsive, professional interface in which every control is connected to a real API.
9. Evaluate the system experimentally: question relevance, retrieval Precision@K and Recall@K, fixed versus adaptive difficulty, AI–human agreement, latency and completion rate.
10. Produce academic documentation: audit, synopsis, architecture, database schema, API reference, methodology, experimental results, testing report and future scope.

4. Technical Description

4.1 Technology stack

Layer	Technology	Role in the system
Frontend	React 19, Vite 6, Tailwind CSS 3, Recharts, Monaco Editor, Axios	Single-page application: landing, authentication, dashboard, resume analysis, interview setup, voice/text interview room, DSA coding workspace, results, history, profile
Backend	Python 3.12, FastAPI, Uvicorn, SQLAlchemy 2 (async), Alembic, Pydantic	REST API, WebSocket endpoint, validation, persistence and migrations
Agents	LangGraph 1.x, LangChain-core	State graph with conditional routing between interview agents, evaluation and difficulty adaptation
LLM providers	Groq (openai/gpt-oss-120b with fallbacks), Google Gemini	Question generation, answer evaluation, resume screening and extraction, report narrative
Retrieval	FAISS (IndexIDMap2 over IndexFlatIP), Gemini gemini-embedding-001 (768-d), local feature-hashing embeddings as offline fallback	Resume chunk storage and cosine-similarity retrieval filtered per candidate
Speech	Deepgram Nova-2 (STT), ElevenLabs multilingual v2 (TTS), Google Translate TTS fallback	Voice interview pipeline over WebSockets
Data	PostgreSQL 16 (production), SQLite via aiosqlite (development/tests), JSON side-car for FAISS metadata	Users, candidates, sessions, round transcripts, interview questions, reports
Documents	PyMuPDF, python-docx, ReportLab	Resume parsing and PDF report rendering
Security	Argon2 password hashing, PyJWT access tokens, rotating refresh-token cookies, rate-limiting middleware, magic-byte upload validation	Authentication, abuse prevention, safe uploads
Testing & experiments	Pytest, pytest-asyncio, Playwright (screenshots), Pandas, scikit-learn, SciPy, Matplotlib	Automated tests, headless UI rendering, experimental evaluation

4.2 System architecture

The system is a two-tier web application. The React single-page application talks to the FastAPI backend through a REST API for transactional operations (authentication, uploads, configuration, reports, analytics) and through a WebSocket for the real-time interview conversation. Inside the backend, the InterviewOrchestrator is the single controller: it owns in-memory session state, per-session locks and database persistence, and it invokes the compiled LangGraph graph once per interview turn. Agents never call each other; routing happens only through the graph's conditional edges. Figure 1 shows the components and their main interactions.

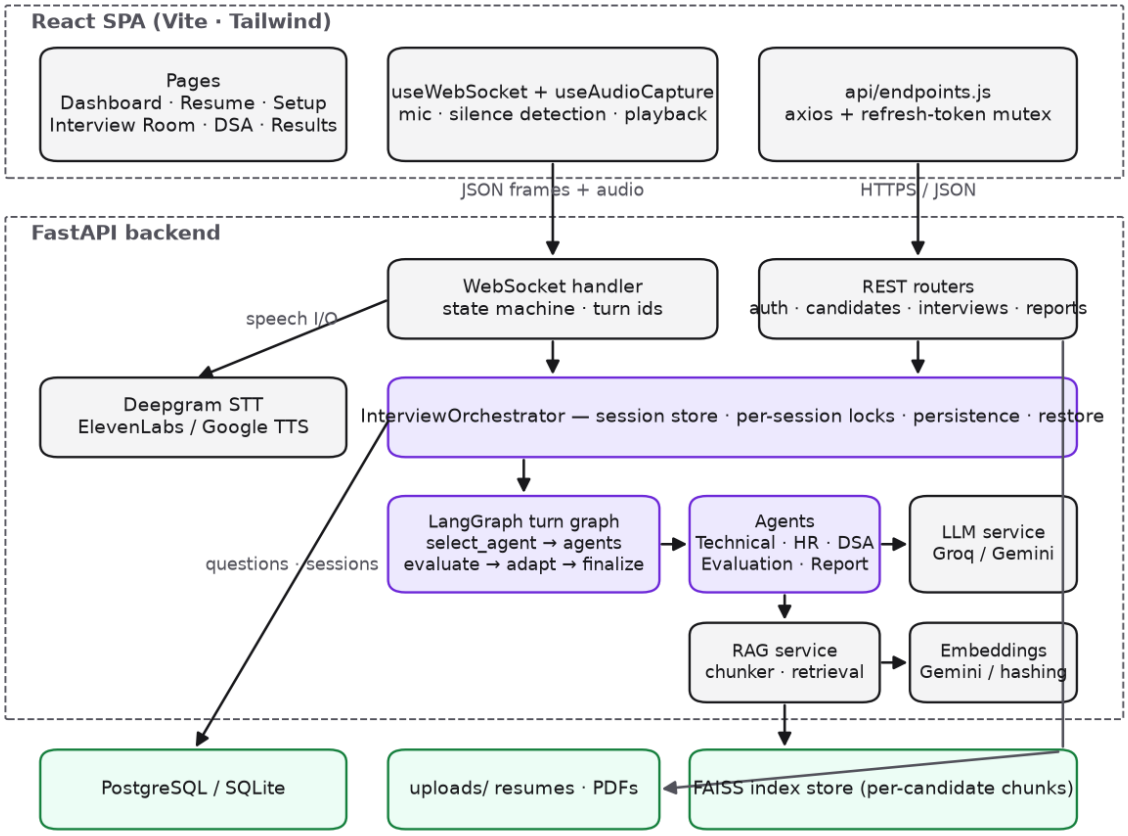


Figure 1. System architecture.

4.3 LangGraph multi-agent workflow

Every interaction with the engine is one invocation of the compiled StateGraph with a shared InterviewState (TypedDict). The entry node routes on the phase of the interaction: "open" selects the round agent that produces the first question; "answer" evaluates the candidate's answer, adapts the difficulty and either asks the next question or concludes the round when the configured question limit is reached; "finalize" aggregates the round, stores round results and either advances to the next round or completes the interview. The DSA coding round runs

through REST endpoints because it needs a code editor, but it shares the same difficulty policy and the same finalisation node.

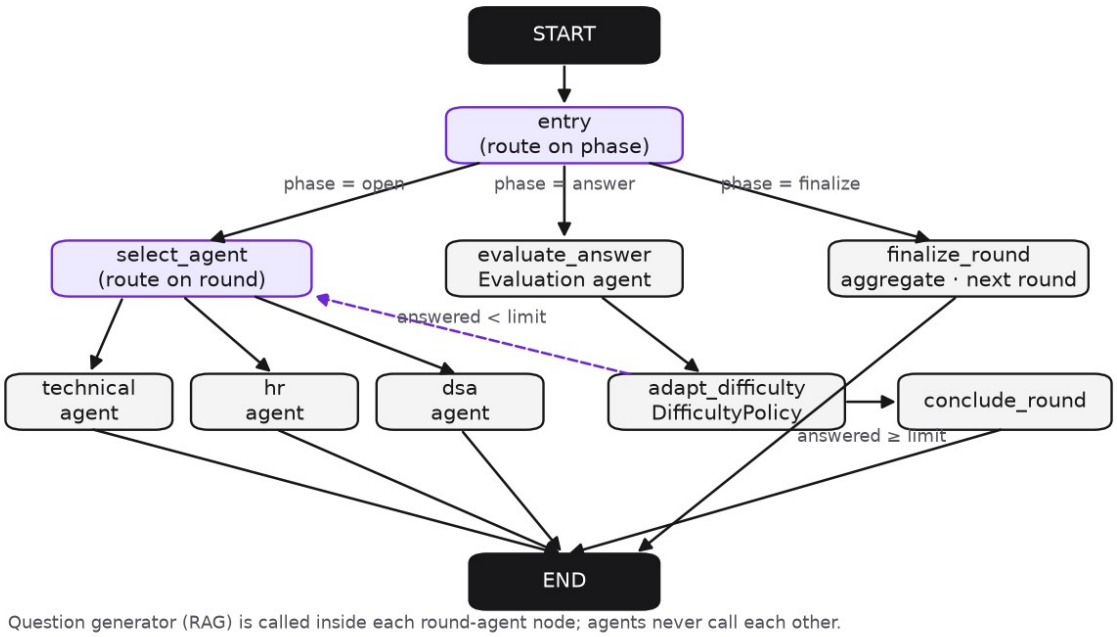


Figure 2. LangGraph interview turn graph (nodes and conditional edges).

Node	Responsibility
entry / select_agent	Routing-only nodes: conditional edges on phase and on current round
technical_agent, hr_agent, dsa_agent	Persona framing and a call to the question generator with retrieved resume context; append the question record; add a spoken introduction on the first question
evaluate_answer	Evaluation agent scores the pending question against the rubric with evidence quotes; stores scores, gaps, ideal-answer points, learning topics and provenance
adapt_difficulty	Applies the DifficultyPolicy to the score to set the difficulty for the next question (no-op when adaptivity is disabled for the session)
conclude_round	Generates the closing sentence once the question limit is reached
finalize_round	Aggregates evaluations into a 0–100 round score with rubric averages, strengths, weaknesses and learning topics; computes filler-word communication analytics; advances the round queue or completes the interview and computes the overall score

4.4 Resume intelligence and RAG pipeline

Uploaded resumes are validated (extension, size and magic bytes), parsed with PyMuPDF or python-docx, screened against the target role, and converted by the LLM into a structured

profile with name, skills, projects, experience, education and summary. A deterministic section parser provides a fallback profile when the LLM is unavailable, and in both paths only text that occurs in the resume is emitted. For retrieval, headings are detected with regular expressions for the standard sections (summary, skills, projects, experience, education, certifications) and each section is split into overlapping windows of 600 characters with 120-character overlap on bullet and sentence boundaries. Each chunk is tagged with its section and carries candidate_id, section, chunk_index and character offsets as metadata. Vectors are L2-normalised and stored in a FAISS IndexIDMap2 over an inner-product index, giving cosine similarity and the ability to delete or replace one candidate's chunks without re-embedding the whole collection. Queries are filtered by candidate before the top-k cut, so one candidate can never retrieve another candidate's data.

The question generator builds a retrieval query from the planned category, its keywords, the target role and the last answer, formats the retrieved chunks as a context block with citation tags, and asks the LLM for a JSON object containing the question, whether it is a follow-up, a rationale and the source tags used. Near-duplicate questions (sequence-matcher ratio ≥ 0.8) trigger regeneration; when the LLM is unavailable a curated bank supplies a question that is labelled generation_source = "bank".

4.5 Evaluation rubric and adaptive difficulty engine

The Evaluation agent scores each answer from 0 to 10 on four criteria and computes the weighted overall score:

Criterion	Weight	What it measures
Technical correctness	40%	Factual and technical accuracy of what was actually said
Conceptual understanding	25%	Whether the candidate understands why, not only what
Answer completeness	20%	Coverage of the key points the question requires
Communication clarity	15%	Structure and clarity of the explanation (never a proxy for knowledge)

The evaluator must quote evidence from the answer, list gaps and ideal-answer points, and suggest learning topics. Non-answers ("I don't know") score zero and are marked unavailable. When the LLM is unavailable a lexical heuristic estimates completeness and clarity from length and structure, caps technical and conceptual scores at 6, and flags the result as heuristic; the interface and the PDF display this flag. The DifficultyPolicy maps the overall score to the next level on the ladder easy → medium → hard: score ≤ ADAPTIVE_DECREASE_MAX (default 4) steps down, score ≥ ADAPTIVE_INCREASE_MIN (default 8) steps up, otherwise the level is held, clamped at both ends. Thresholds are environment settings and adaptivity can be switched off per session, which is how the fixed-difficulty baseline of Experiment 3 is obtained.

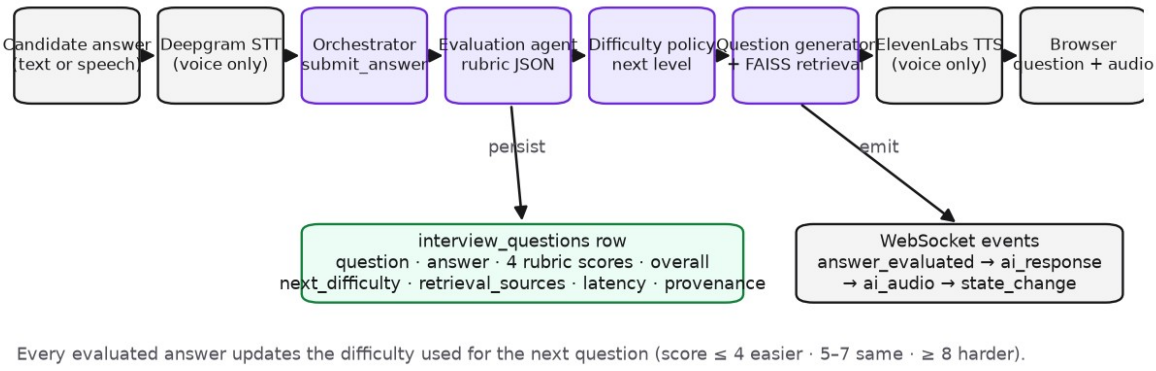


Figure 3. Data flow for one interview turn.

4.6 Real-time voice pipeline

The browser captures microphone audio with MediaRecorder (WebM/Opus, 400 ms chunks) and detects silence from the RMS amplitude. Chunks are sent as base64 frames over the WebSocket; on silence_detected the server transcribes the buffered audio with Deepgram Nova-2, runs the orchestrator turn, emits the evaluation and the next question as text immediately, synthesises speech with ElevenLabs (MP3, Google TTS fallback) and streams it to the browser. Every interviewer turn carries a turn_id; the client plays audio sequentially, ignores duplicate frames and sends playback_finished so the server switches to listening at the right moment. A per-session asyncio lock serialises turns, and text input is available in every state so the interview never stalls when voice services are unavailable. The interview states are initializing, listening, processing, generating, speaking, waiting, completed and error.

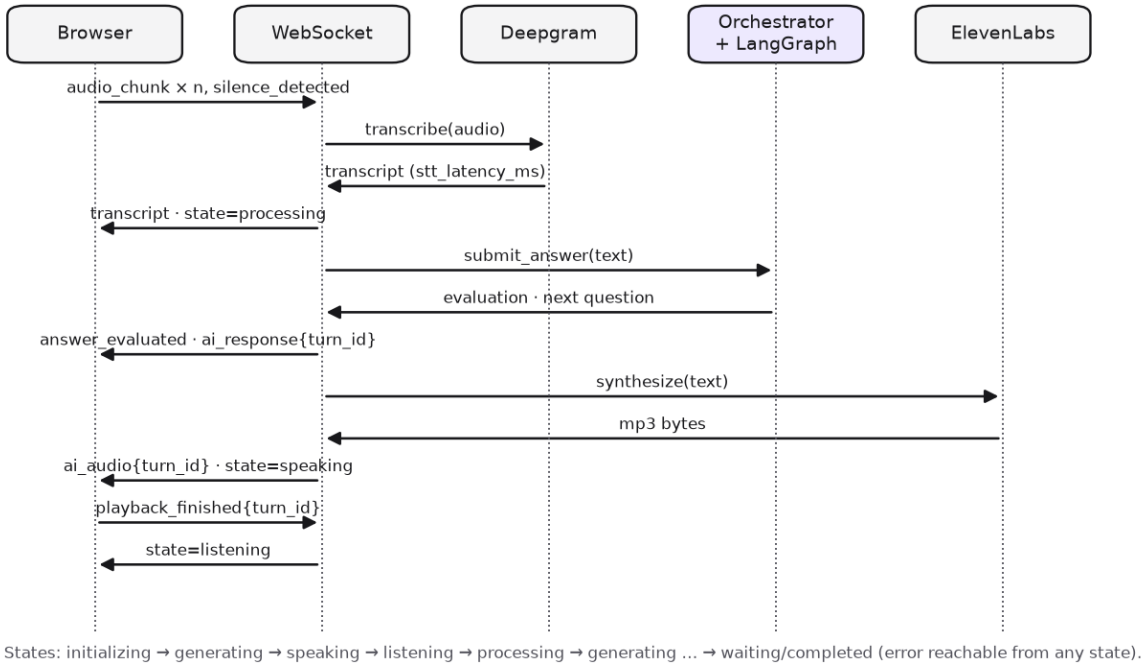


Figure 4. Voice interaction sequence.

4.7 Database design

The relational schema (Figure 5) stores users, refresh tokens, candidate profiles, interview sessions with their configuration and persona, round transcripts, one row per asked question with its rubric scores, difficulty trace, retrieval sources and latencies, and generated reports. Candidates are unique per (user, e-mail), which also fixed a defect in the prototype where one user could take over another user's candidate record. Alembic migrations create the schema on PostgreSQL; SQLite is used for development and automated tests.

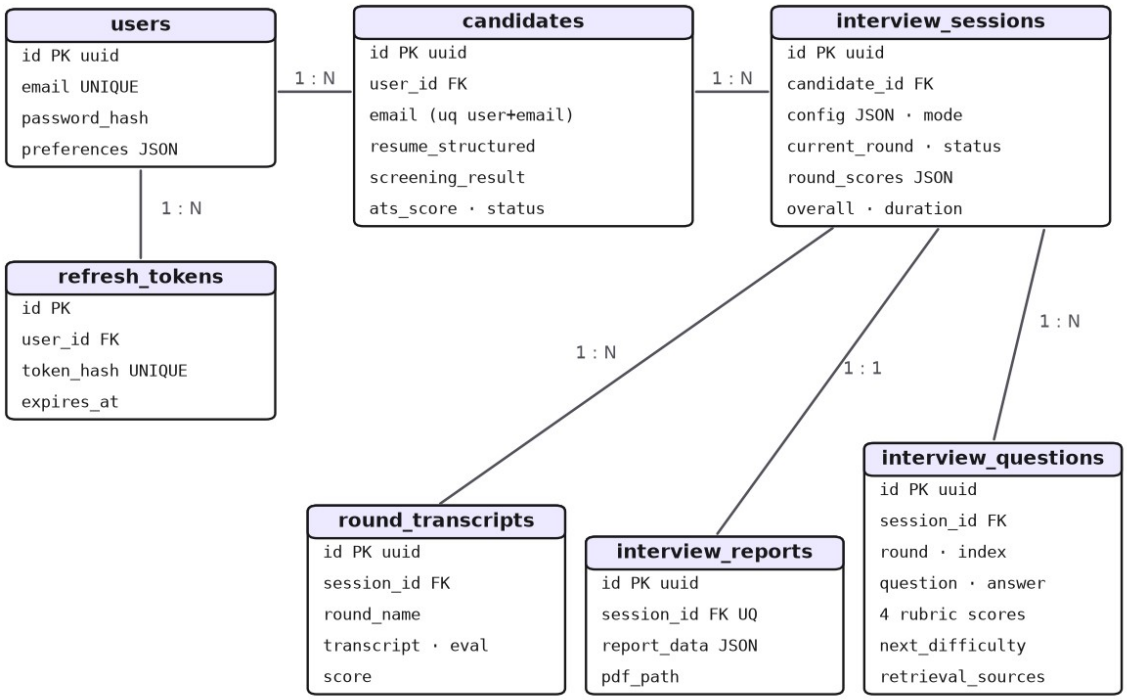


Figure 5. Entity–relationship diagram.

4.8 API and WebSocket protocol

The REST API is grouped into authentication (signup, login, refresh-token rotation, profile, preferences, account deletion), candidates (upload, profile dashboard, RAG inspector: chunks and retrieval test), interviews (configuration options, start with configuration, history, details with questions, cancel, engine graph description), the DSA round (problem, hint, submit), reports (per-session JSON and PDF), dashboard analytics and a system-status endpoint that reports which AI services are live. The WebSocket protocol defines client messages (start_round, audio_chunk, silence_detected, candidate_text, playback_finished, end_round, cancel_interview) and server messages (connected, session_history, state_change, ai_response, ai_audio, transcript, answer_evaluated, round_complete, interview_complete, error). Full details are in the repository document API_DOCUMENTATION.md.

4.9 User interface

The React interface was rebuilt as a design system of reusable components (buttons, cards, inputs, badges, progress meters, score rings, tabs, modals, tooltips, loading, empty and error states) on a dark palette, with an application shell (sidebar navigation and a capability banner that states honestly when AI or voice services are not configured). The pages are: landing, sign-in/sign-up/password recovery, dashboard, resume analysis, interview configuration, interview room, DSA coding workspace, results, history and profile/settings. Every displayed number comes from the API; no page uses simulated data. Screenshots are provided in Annexure A.

5. Methodology

5.1 Development methodology

The work followed an audit-first, incremental methodology on an existing codebase. First, every module of the prototype was read and its baseline measured (39 tests passing with one failing, a broken frontend build on the available Node version, no local database). Findings and a prioritised checklist were recorded in PROJECT_AUDIT.md. Second, correctness and data-integrity defects were fixed and the application was made runnable without external services. Third, the three technical contributions were implemented: resume intelligence with RAG, the LangGraph adaptive engine with rubric evaluation, and persistence with reporting. Fourth, the frontend was redesigned against the real API. Finally, the system was verified by automated tests and live runs, evaluated experimentally and documented. The application remained runnable after each phase and the test suite was executed after every change set.

5.2 Audit findings that shaped the design

Finding in the prototype	Resolution
LangGraph graph compiled but never invoked; rounds advanced by hand-written code	Turn graph with entry routing, agent selection, evaluation, adaptation and finalisation nodes; one ainvoke per turn
Resume stored as a single vector; no retrieval during interviews	Section-aware chunking, per-candidate FAISS retrieval with citations, RAG inspector in the UI
Fixed difficulty by question count	Rubric-driven DifficultyPolicy with configurable thresholds and per-session switch
Fabricated fallback scores (e.g. 85/100, “recommend hire”, “code correct”) when the LLM failed	Flagged heuristic or unavailable evaluations; provenance stored and displayed everywhere
Candidate records keyed by e-mail globally (cross-user takeover possible)	Uniqueness per (user, e-mail); ownership checks on every route
No per-question persistence; reports keyed by candidate	interview_questions and interview_reports tables; per-session reports
Fake telemetry and timed “analysis phases” in the UI	Removed; real upload progress and real evaluation data only
Query-string tokens accepted on every route	Accepted only on PDF download routes

5.3 Experimental design

Five experiments were designed to measure the contributions. All scripts record the runtime environment (LLM availability, embedding backend, thresholds) so that results obtained in degraded mode are never confused with live-model results.

#	Research question	Design	Metrics
1	Does resume grounding make questions more relevant?	Five synthetic candidates (ML, backend, frontend, data engineering, DevOps), five technical questions each, under generic, structured-profile and RAG generation	Resume-term overlap, entity-mention rate, citation rate
2	How good is retrieval?	18 labelled queries (query → relevant resume section) over the five resumes; section-aware chunking versus naive 300-character windows	Precision@K and Recall@K for K = 1..3, cross-candidate leakage, latency
3	Does adaptive difficulty place candidates appropriately?	200 simulated candidates with latent ability $\theta \sim U(1.5, 9.5)$, six questions each, observed score $N(\theta - \text{penalty(level)}, 1.2^2)$, run through the production DifficultyPolicy versus fixed easy/medium/hard	Spearman ρ between ability and final score, mean absolute error, share of questions at the target level
4	Do AI scores agree with human raters?	Human ratings CSV (template provided) scored by the Evaluation agent	Pearson r, Spearman ρ, MAE, quadratic-weighted Cohen's κ per criterion
5	What are the latencies and completion rate?	Persisted generation and evaluation latencies and session statuses from the application database; STT latency from server logs	Mean, p50, p95 in ms; completion rate

5.4 Testing methodology

Unit tests cover the difficulty policy (thresholds, clamping, disabling), the evaluation agent (LLM path, heuristic fallback, non-answers, aggregation), the question generator (category planning, de-duplication, RAG sources, bank fallback), the chunker, the hashing embeddings, the FAISS store (persistence, backend guard, filtered search) and the RAG service (isolation, re-indexing, deletion). Graph-level tests drive the LangGraph through open → answer → adapt → conclude → finalize. API tests cover validation, security headers, rate limiting, uploads and account deletion. A WebSocket integration test runs a complete text interview through the real orchestrator with persistence and reconnection. A live smoke script exercises the full HTTP and WebSocket flow against a running server with the production LLM. The frontend is linted and built, and every page was rendered headlessly with Playwright at desktop and mobile widths for visual review.

6. Outcomes

6.1 Delivered system

All acceptance criteria of the project brief are met: a user can register and sign in; upload a resume and see the extracted profile; retrieval returns relevant resume context with citations; the candidate configures role, rounds, difficulty, duration, mode and question count; the multi-agent system conducts DSA, Technical and HR rounds with personalised questions; voice and text interviews run end to end; answers are evaluated and difficulty adapts; everything is persisted; dashboards show actual results; PDF reports are generated; the interface is consistent and responsive; and the test suite passes.

6.2 Verification results

Check	Result
Backend automated tests (pytest)	57 passed, 0 failed (baseline before the project: 39 passed, 1 failed)
Frontend lint (ESLint) and production build (Vite 6)	0 findings; build succeeds with 15 lazily loaded page chunks
Live end-to-end smoke run with Groq LLM and Gemini embeddings	All 19 steps passed: signup, upload (LLM screening, 6 chunks indexed), retrieval, configuration, DSA problem/hint/submit (LLM evaluated), WebSocket technical round with adaptive difficulty, history, dashboard, JSON report, PDF report; 0 server errors
Live end-to-end smoke run in degraded mode (no API keys)	All steps passed with bank questions and flagged heuristic scoring
Headless rendering of all pages (Playwright, 1440 px and 390 px)	0 console errors; layout defects found and fixed (clipped chart labels, misaligned card header, overlapping sticky bar)

6.3 Experiment 1 — generic versus resume-based question generation

Table 6.1 reports the relevance metrics for five candidates and five questions per condition. The runs recorded here were executed in degraded mode (question bank, local embeddings); with a live model the RAG condition additionally rewrites the question text from the retrieved context.

Condition	Resume-term overlap	Entity-mention rate	Citation rate
Generic (role only)	0.032	0.00	0.00
Structured profile, no retrieval	0.114	0.48	0.00
Structured profile + RAG	0.110	0.48	1.00

Resume grounding raises the share of question tokens that occur in the resume by a factor of 3.5 and makes 48% of questions name a real project, company or role, compared with none in the generic condition. The RAG condition attaches a resume citation to every question.

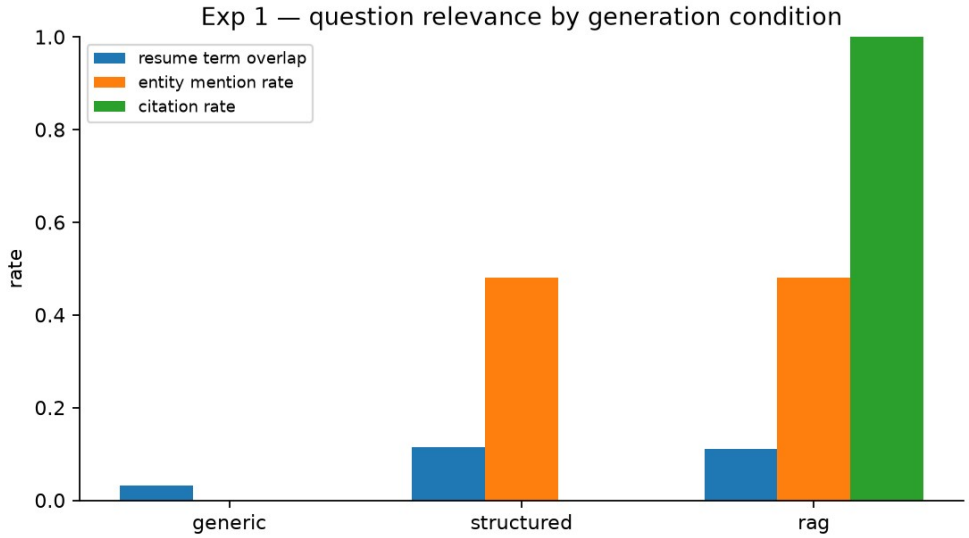


Figure 6. Experiment 1 — question relevance by generation condition.

6.4 Experiment 2 — retrieval quality

Chunking	P@1	P@2	P@3	R@1	R@2	R@3	Leakage	Latency (mean / p95)
Section-aware (ours)	0.944	0.639	0.426	0.778	0.944	0.944	0 hits	2.4 ms / 3.3 ms
Naive 300-char windows	0.333	0.278	0.222	0.250	0.444	0.528	0 hits	2.5 ms / 3.4 ms

Section-aware chunking almost triples Precision@1 and reaches Recall@2 of 0.94 even with the lexical hashing backend; per-candidate filtering produced zero cross-candidate leakage across 54 returned hits. With the Gemini embedding backend in the live run, a query for “CNN PyTorch accuracy” returned the correct projects chunk with similarity 0.61 in 515 ms (including the embedding API call).

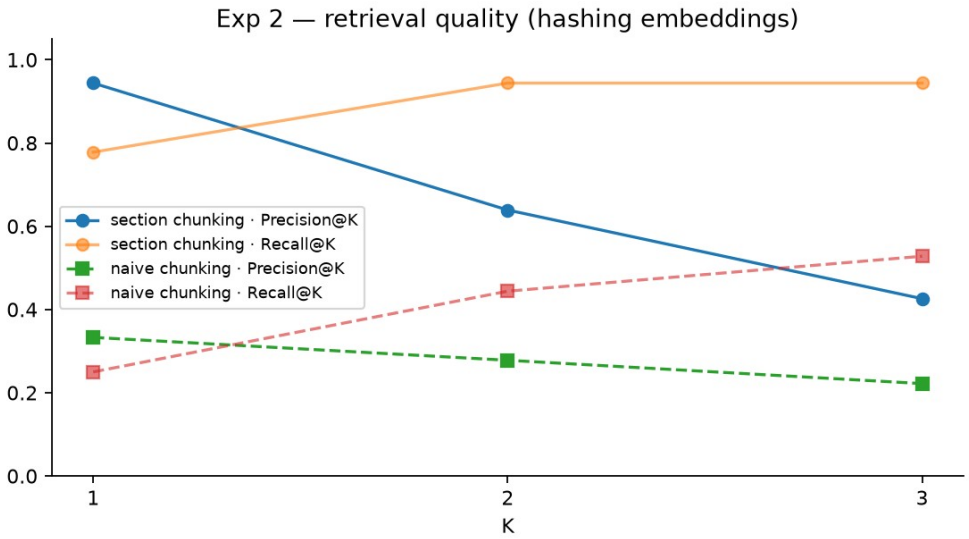


Figure 7. Experiment 2 — Precision@K and Recall@K for both chunkers.

6.5 Experiment 3 — fixed versus adaptive difficulty

Condition	Spearman ρ (ability vs. score)	Mean absolute error	Time at target level
Adaptive (start medium)	0.978	0.436	0.707
Fixed medium	0.979	0.379	0.290
Fixed easy	0.978	0.513	0.485
Fixed hard	0.978	0.919	0.225

Under this noise model all policies rank candidates almost perfectly, so adaptivity does not change ranking. Its measured benefit is appropriateness: adaptive interviews spend 71% of questions at the level matching the candidate's ability against 29% for a fixed-medium interview, while keeping the error comparable to the best fixed level and far below fixed-hard. Fixed-medium has the lowest error because the simulated score is centred on the ability at medium difficulty by construction; this is a property of the simulation, not evidence against adaptivity.

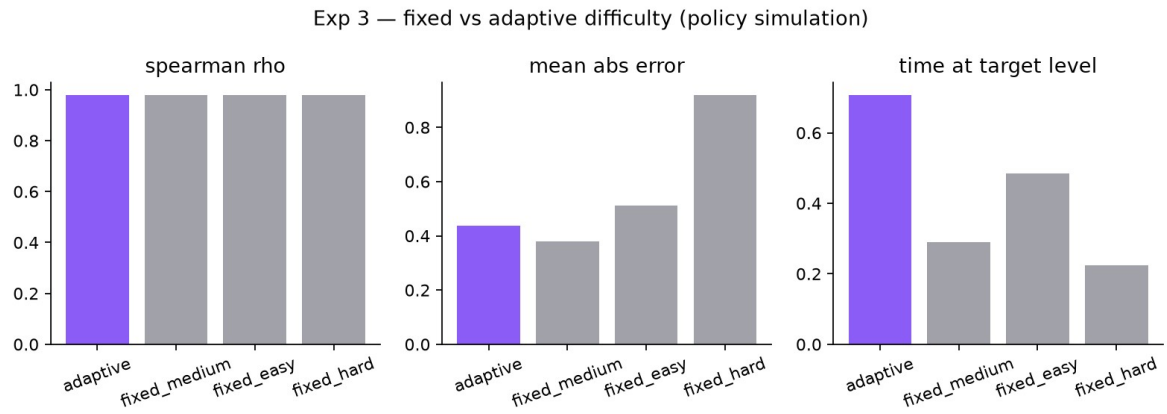


Figure 8. Experiment 3 — fixed versus adaptive difficulty (policy simulation).

6.6 Experiment 4 — AI versus human agreement

This experiment is implemented but not yet run: it requires answers rated by human evaluators. A template CSV (question, answer and five human scores) and the script computing Pearson r, Spearman ρ, mean absolute error and quadratic-weighted Cohen's κ per criterion are provided. No synthetic human ratings were generated, and no agreement figure is claimed.

6.7 Experiment 5 — latency and completion

Measured from the development database after live runs. With the production LLM, question generation and evaluation are network-bound; the degraded path is sub-millisecond. The measurement script reports mean, median and 95th percentile latencies split by evaluation source, the sessions by status and the interview completion rate, and reads speech-to-text latency from server logs when voice is used. In the recorded run one session was completed out of one started (completion rate 1.00, n = 1); the figure demonstrates the measurement pipeline rather than a population estimate.

6.8 Qualitative outcome

In the live run with the production model the first technical question for an ML Engineer candidate was grounded in the resume's Callback Rehearsal project ("what trade-offs did you consider when choosing FAISS over other vector stores, and how would you scale the retrieval layer?") with citations projects#3 and experience#4; an off-topic answer received technical correctness 2.0 and the difficulty dropped to easy; the next question moved to the candidate's Traffic Sign Classifier project; the report narrative, strengths, weaknesses, recommendations and ten learning topics were generated from the evaluations and exported to PDF.

7. Knowledge / Skill Improvement

- **Agentic AI engineering.** Designing a LangGraph state graph with typed shared state, conditional routing and per-turn invocation; separating routing from agent behaviour; restoring agent state from a database after restarts.
- **Retrieval-Augmented Generation.** Section-aware chunking, metadata design, FAISS index types (IndexIDMap2, inner-product on normalised vectors), per-tenant filtering, citation formatting, and evaluation with Precision@K and Recall@K.
- **LLM evaluation design.** Writing rubric prompts that demand evidence, enforcing JSON output with repair, aggregating per-answer scores, and designing honest fallbacks with provenance instead of silent defaults.
- **Adaptive assessment.** Implementing and simulating a threshold policy, understanding its relation to computerised adaptive testing and item-response theory.
- **Real-time systems.** WebSocket protocol design with explicit state machines, turn identifiers, locks against concurrent turns, client-confirmed playback, and graceful degradation when speech services are missing.
- **Backend engineering.** Async SQLAlchemy, Alembic migrations that run on both PostgreSQL and SQLite, idempotent startup migrations, JWT with refresh-token rotation, rate limiting, secure file validation.
- **Frontend engineering and design.** Building a reusable component kit and design tokens with Tailwind, data-driven dashboards with Recharts, a Monaco-based coding workspace, and iterating on visual quality from headless screenshots.
- **Experimental method.** Designing measurable experiments, recording the environment with every result, and reporting negative or inconclusive findings honestly.
- **Software quality.** Auditing a legacy codebase, writing unit, integration and end-to-end tests, and maintaining documentation as part of the deliverable.

8. Applications

- **Placement preparation.** Students can rehearse technical and behavioural interviews on their own resumes and receive rubric-based feedback and learning topics before campus placements.
- **Training and placement cells.** Institutions can host the platform to give cohorts structured practice and track readiness trends over time.
- **Recruiter screening support.** The ATS screening, structured profile and interview report can support first-round screening, with the provenance flags making the basis of each score transparent.
- **Corporate learning.** The adaptive engine and rubric can be reused for internal skill assessments by changing the question categories and job description.
- **Research test-bed.** The persisted question-level data (retrieval sources, difficulty trace, latencies) supports studies of LLM question quality, evaluator reliability and adaptive policies; the human-rating template enables agreement studies.
- **Reusable components.** The RAG service, LangGraph orchestration pattern and voice pipeline are independent modules that can be adopted in other conversational-AI projects.

9. Conclusions

The project transformed a partially working prototype into a complete, verified and documented AI-powered mock interview platform. Its three technical contributions are: (1) resume-aware, retrieval-augmented question generation with per-candidate FAISS retrieval and source citations; (2) a LangGraph-based multi-agent interview engine in which a single orchestrator routes between DSA, Technical, HR and Evaluation agents and adapts difficulty from rubric scores; and (3) an evaluation and analytics layer that scores every answer on a standard rubric with evidence, persists everything at question level, and renders dashboards and PDF reports.

Experimentally, section-aware chunking tripled retrieval Precision@1 over naive windows (0.94 versus 0.33) with no cross-candidate leakage, resume grounding raised the share of questions that name the candidate's real work from 0% to 48% even in bank mode, and the adaptive policy placed 71% of questions at the candidate's ability level versus 29% for a fixed policy in simulation. The system is verified by 57 automated tests, lint-clean and buildable frontend code, and live end-to-end runs both with the production LLM and in a clearly flagged degraded mode.

Equally important is what the system no longer does: it does not invent scores when an AI service fails, it does not display simulated metrics, and it does not let one user reach another user's data. These corrections make the results trustworthy enough to be used for real preparation and for research.

10. Limitations

1. Candidate code is not executed; DSA submissions are judged by LLM static reasoning, which can misjudge subtle bugs.
2. Answer scoring is LLM-as-judge; its reliability depends on the configured model. The human-agreement study (Experiment 4) has not yet been run because it requires manually rated answers.
3. Experiments 1 and 2 were recorded with the lexical hashing embedding backend and bank questions; live-model results may differ and should be regenerated with API keys configured.
4. Experiment 3 is a simulation of the policy, not a study with real candidates; Experiment 5 has a single recorded session.
5. Speech recognition and synthesis are request/response calls rather than streaming, so voice turns carry a few seconds of latency.
6. Orchestrator sessions, practice sessions and debrief conversations live in process memory; interview state is restorable from the database but practice and debrief are not.
7. Password-reset and verification tokens are logged rather than e-mailed; there is no mail service.
8. The local embedding fallback captures vocabulary overlap, not paraphrase; semantic retrieval needs the Gemini backend.

11. Future Directions

- **Sandboxed code execution** with hidden test cases to ground DSA correctness instead of static reasoning.
- **Human-rated calibration.** Collect the human-rating dataset, run Experiment 4, and calibrate or ensemble the evaluator; report per-criterion reliability across raters.
- **Item-response-theory based adaptivity.** Replace threshold rules with a Bayesian ability estimate and learned question difficulty parameters.
- **Hybrid retrieval and re-ranking** (BM25 + dense with a cross-encoder) and indexing of the job description alongside the resume.
- **LangGraph checkpointing and human-in-the-loop** so graph state is persisted natively and a mentor can inject questions.
- **Streaming speech** (Deepgram live, streamed TTS) for sub-second voice turns, with barge-in handling.
- **Scalability.** Redis-backed session store and horizontally scaled WebSocket tier; recruiter and mentor roles with cohort analytics.
- **Studies with real candidates** on learning outcomes of adaptive practice, fairness of evaluation across languages and demographic proxies, and over-grounding (invented experience) in resume-aware questions.

12. References

1. P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.
2. J. Johnson, M. Douze and H. Jégou, “Billion-scale similarity search with GPUs,” IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2021 (FAISS).
3. M. Douze et al., “The Faiss library,” arXiv:2401.08281, 2024.
4. LangChain Inc., “LangGraph documentation,” https://langchain-ai.github.io/langgraph/ (accessed 2026).
5. H. Chase, “LangChain,” https://github.com/langchain-ai/langchain (accessed 2026).
6. S. Ramírez, “FastAPI documentation,” https://fastapi.tiangolo.com/ (accessed 2026).
7. Meta Open Source, “React documentation,” https://react.dev/ (accessed 2026).
8. Google, “Gemini API: Embeddings,” https://ai.google.dev/gemini-api/docs/embeddings (accessed 2026).
9. Groq Inc., “GroqCloud API documentation,” https://console.groq.com/docs (accessed 2026).
10. Deepgram, “Nova-2 speech-to-text API documentation,” https://developers.deepgram.com/ (accessed 2026).
11. ElevenLabs, “Text to Speech API documentation,” https://elevenlabs.io/docs (accessed 2026).
12. F. M. Lord, Applications of Item Response Theory to Practical Testing Problems. Hillsdale, NJ: Lawrence Erlbaum, 1980.
13. H. Wainer et al., Computerized Adaptive Testing: A Primer, 2nd ed. Mahwah, NJ: Lawrence Erlbaum, 2000.
14. J. Cohen, “Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit,” Psychological Bulletin, vol. 70, no. 4, pp. 213–220, 1968.
15. L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.
16. A. Vaswani et al., “Attention Is All You Need,” Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
17. C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval. Cambridge University Press, 2008 (Precision@K, Recall@K).
18. ReportLab, “ReportLab PDF Toolkit User Guide,” https://docs.reportlab.com/ (accessed 2026).

19. SQLAlchemy authors, “SQLAlchemy 2.0 documentation,” https://docs.sqlalchemy.org/ (accessed 2026).
20. Tailwind Labs, “Tailwind CSS documentation,” https://tailwindcss.com/docs (accessed 2026).

Annexure A — Application Screenshots

Screenshots were rendered headlessly from the running application at 1440 px width with real data from a live run.

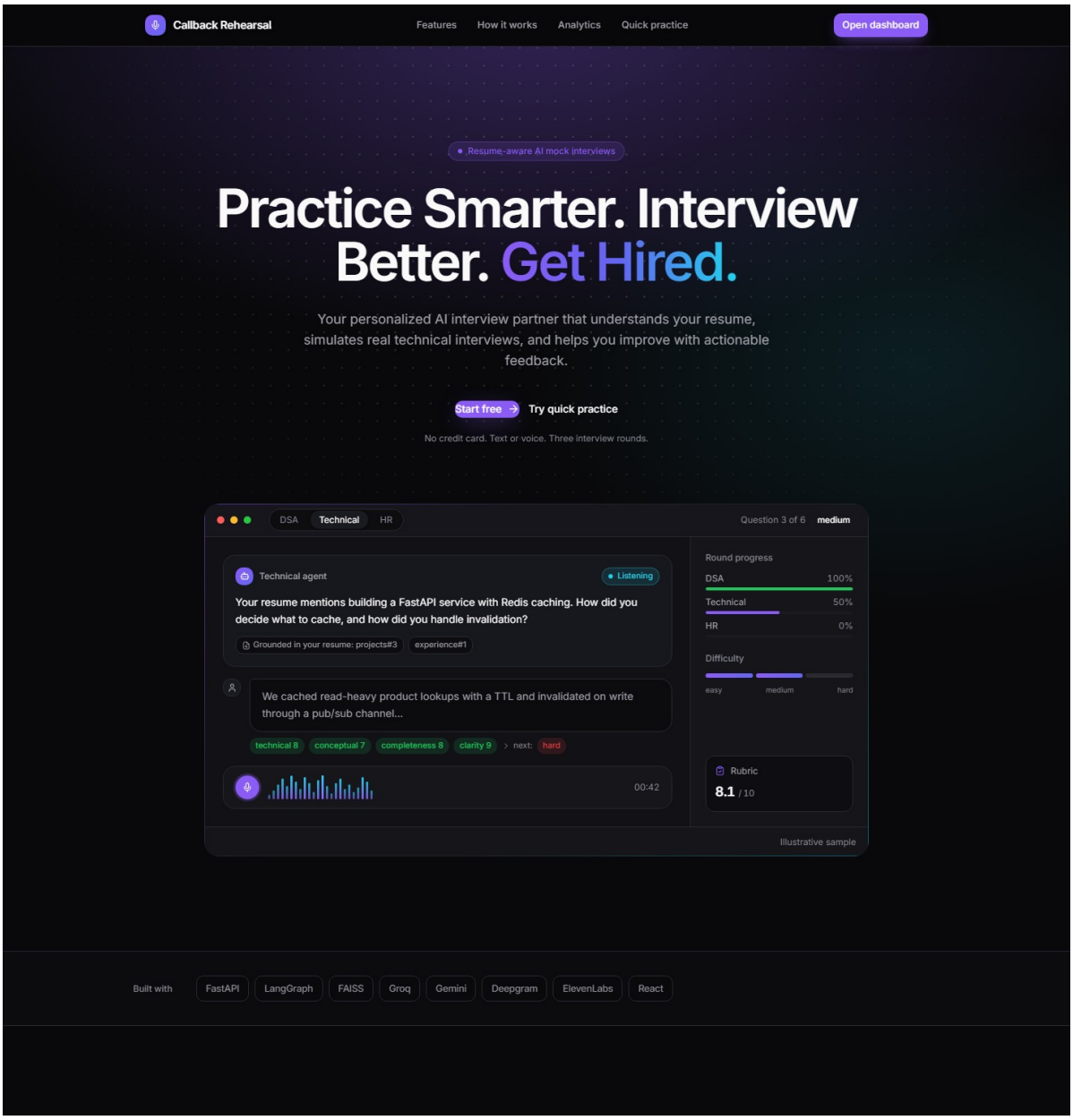


Figure A.1. Landing page.

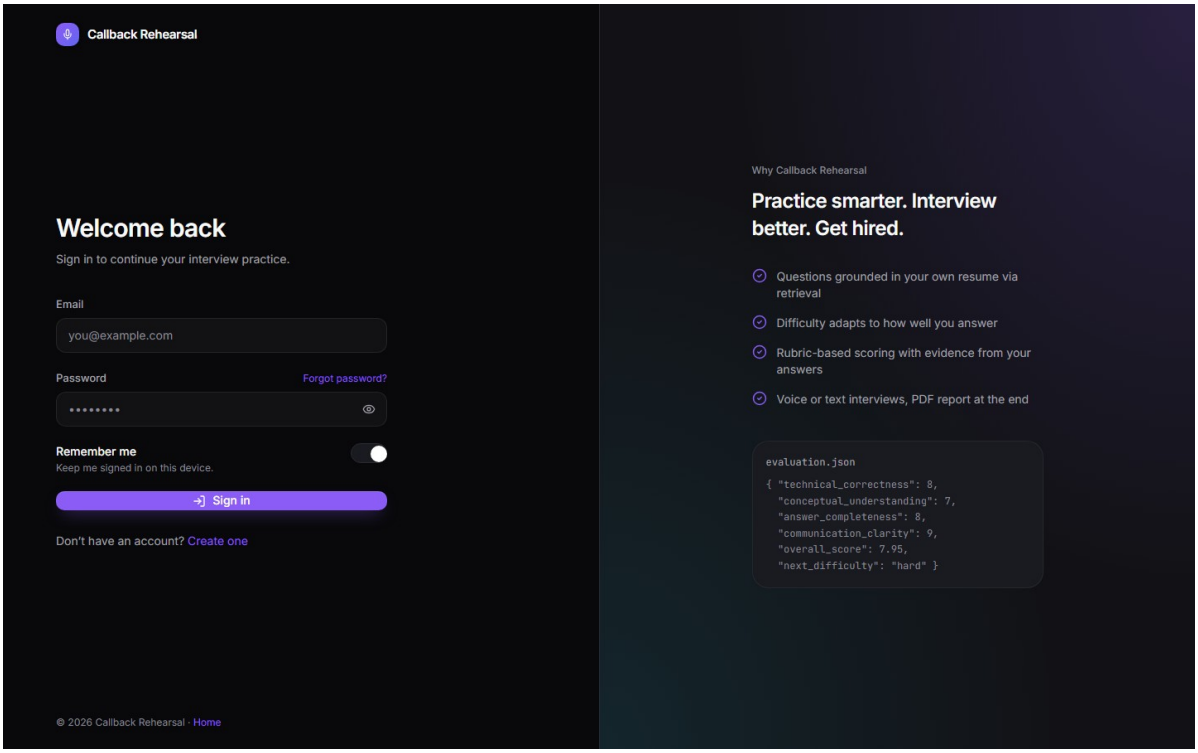


Figure A.2. Sign-in page.

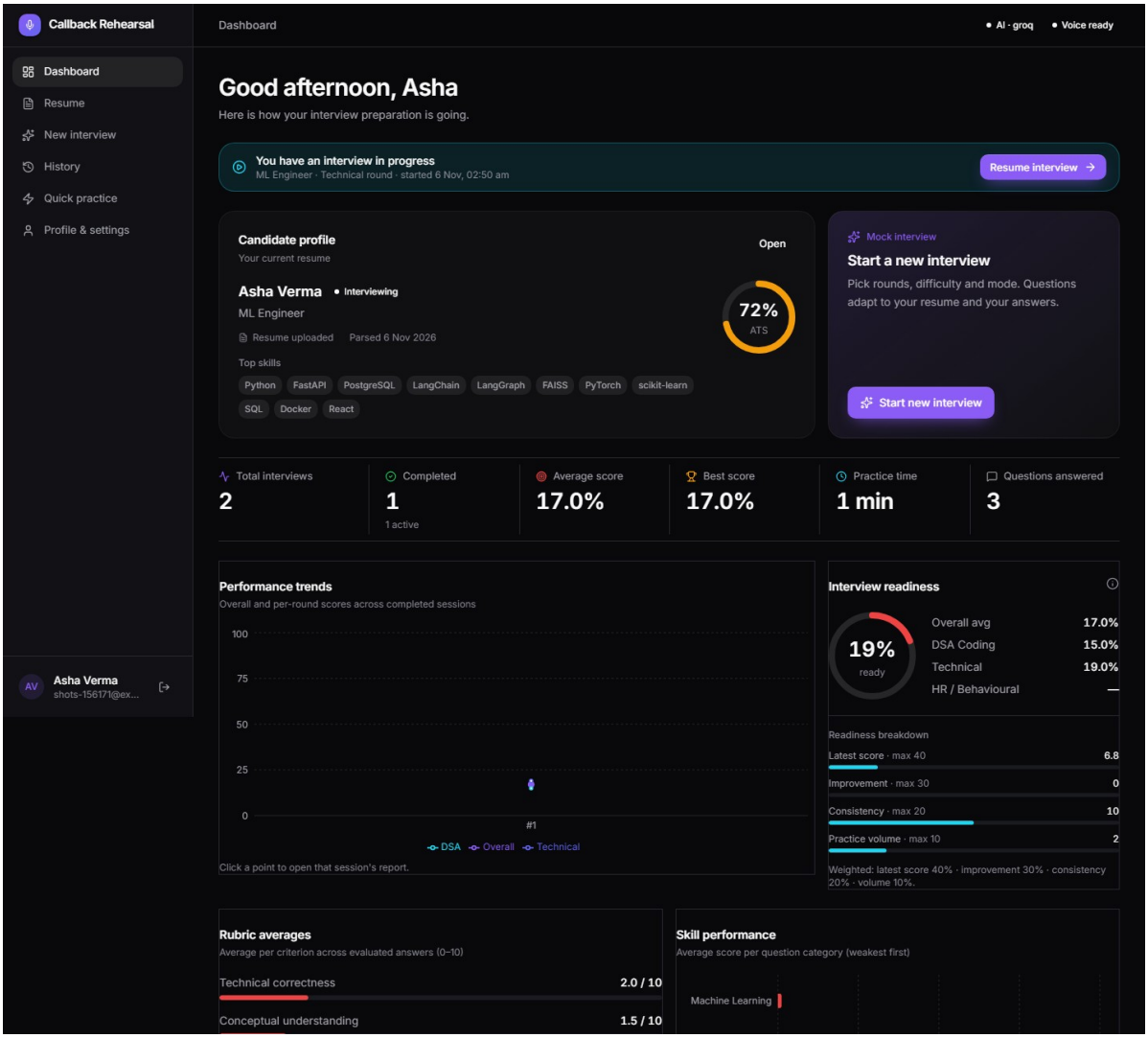


Figure A.3. Dashboard with candidate profile, statistics, trends, readiness and rubric averages.

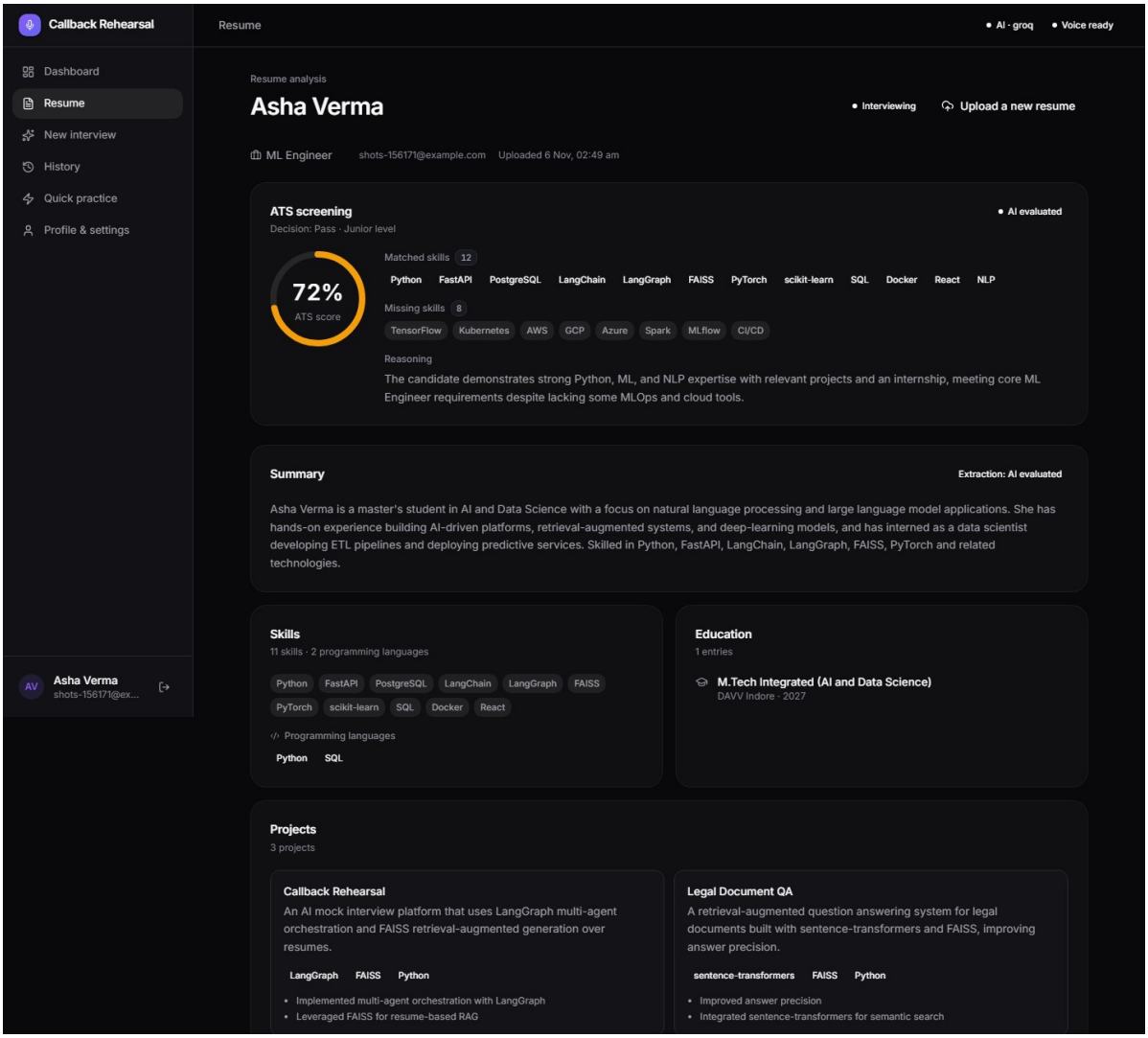


Figure A.4. Resume analysis: ATS screening, structured profile, document statistics and RAG inspector.

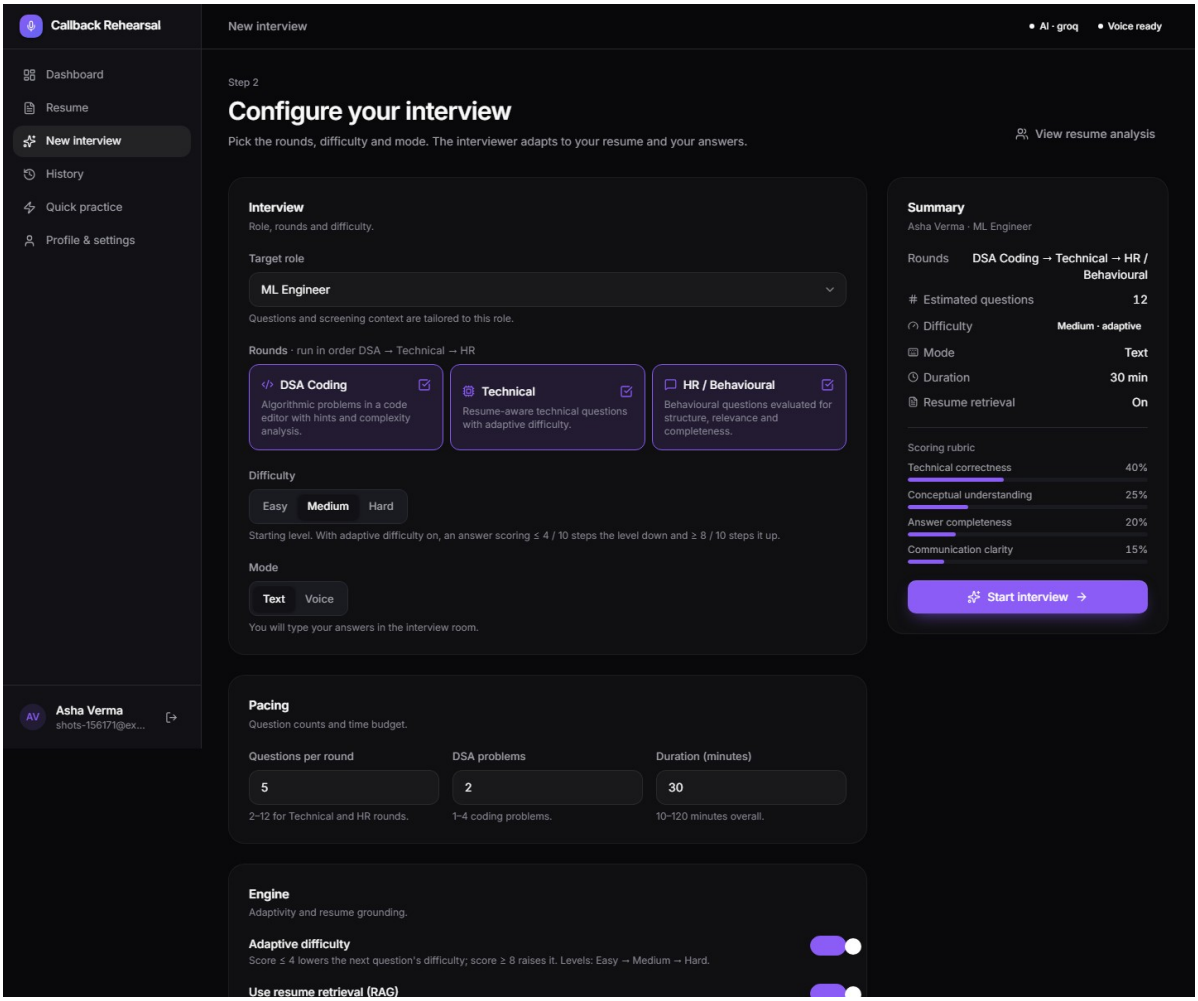


Figure A.5. Interview configuration.

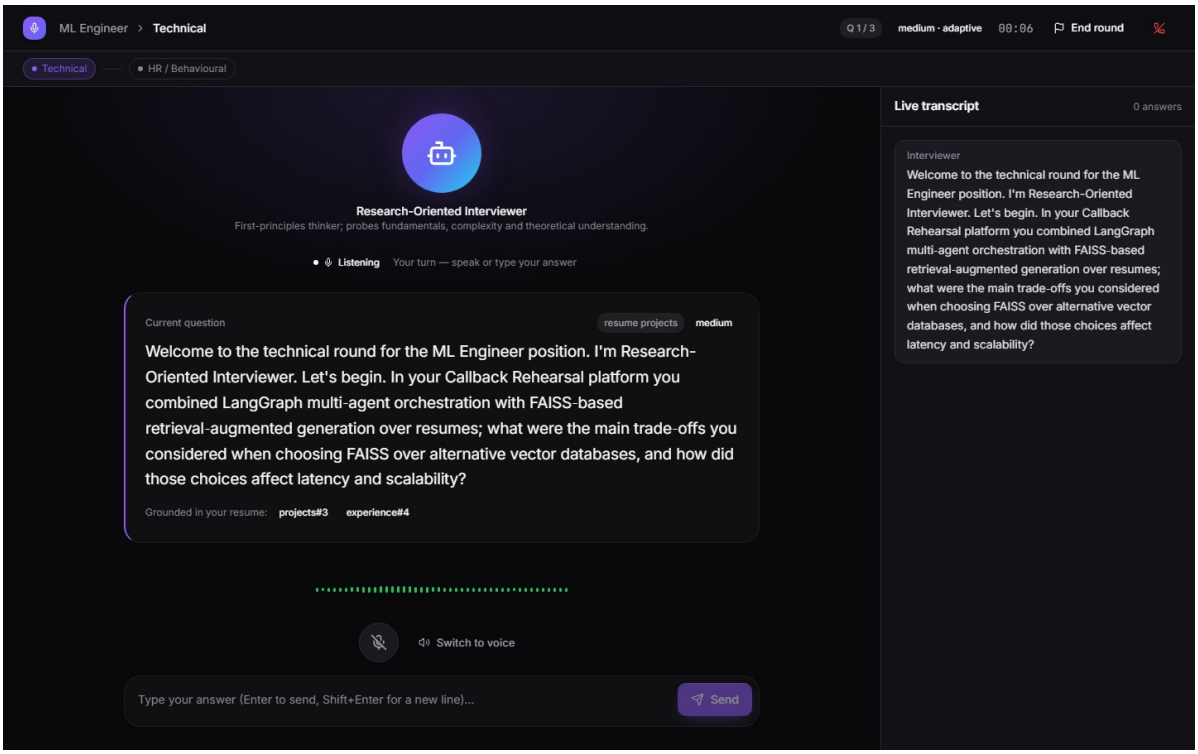


Figure A.6. Interview room with a resume-grounded question, state indicator, waveform and live transcript.

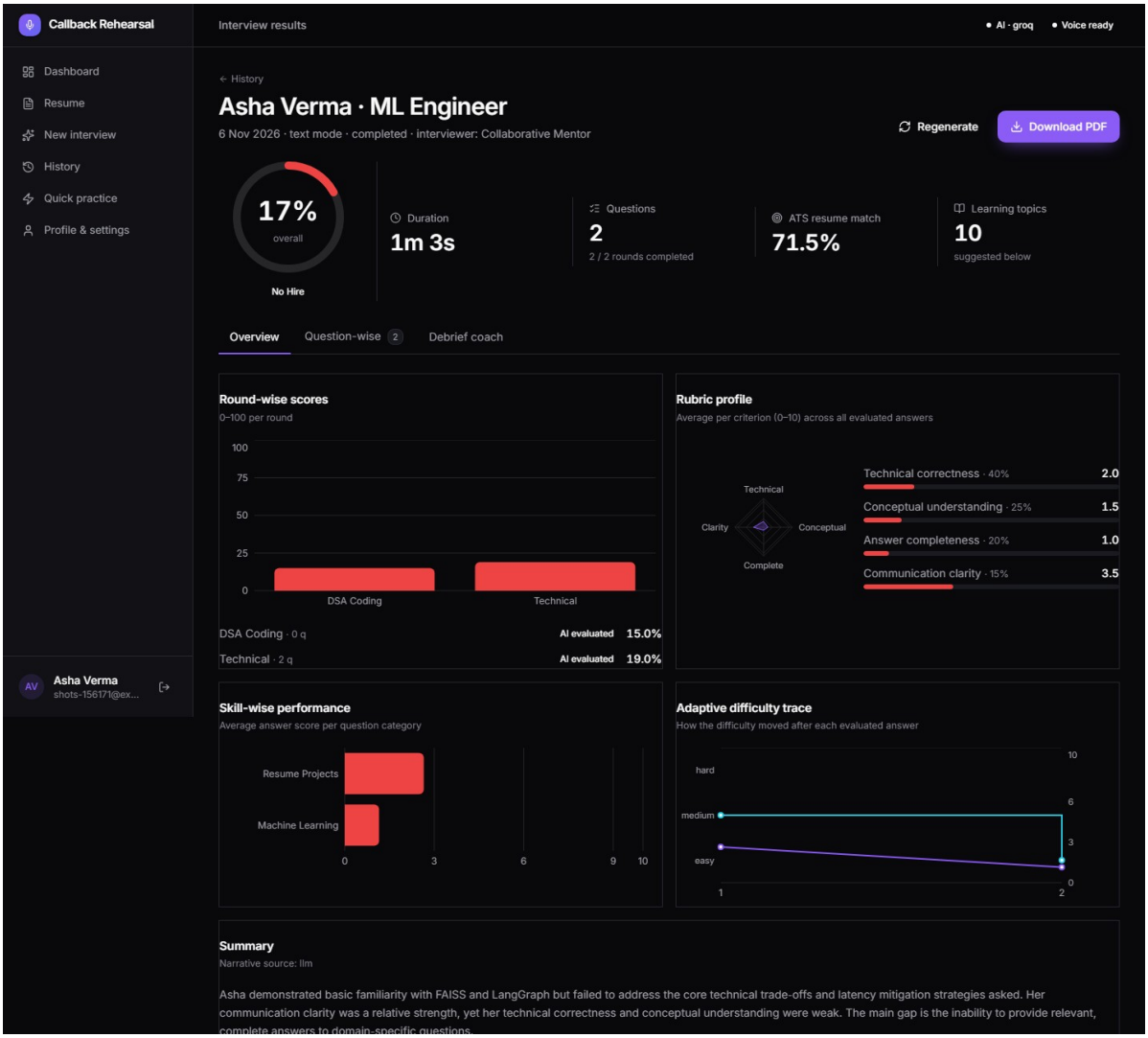


Figure A.7. Results: overall score, round-wise and rubric profile, skill performance, adaptive difficulty trace and narrative.

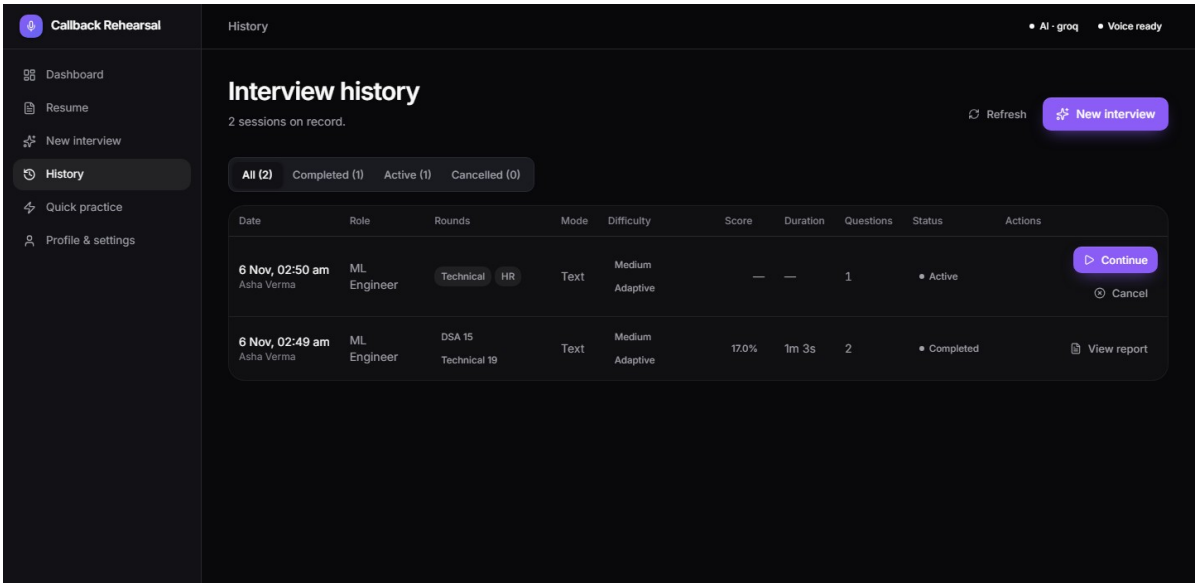


Figure A.8. Interview history.

Annexure B — Repository Layout and Reproduction

The source code is organised as follows:

```
backend/
  main.py        app factory, startup validation, migrations, routers
  core/          config · database · models · schemas · security · timeutil
  agents/        orchestrator · interview_graph · question_generator · evaluation_agent
                 difficulty_policy · tech_agent · hr_agent · dsa_agent · report_agent
  services/      llm_service · faiss_service · rag_service · resume_chunker · resume_parser
                 embeddings/ (gemini, hashing) · stt/deepgram · tts/elevenlabs · rate_limiter
  api/           websocket.py · routes/ (auth, candidates, interviews, dsa, reports, dashboard)
  alembic/       database migrations
  tests/         57 pytest tests
  experiments/   exp1..exp5 scripts, run_all.py, results/
frontend/src/
  App.jsx · api/ · context/ · hooks/ · components/ (ui, layout) · pages/
docs/ (repository root): README, PROJECT_AUDIT, PROJECT_SYNOPSIS, SYSTEM_ARCHITECTURE,
  DATABASE_SCHEMA, API_DOCUMENTATION, METHODOLOGY, EXPERIMENTAL_RESULTS, TESTING_REPORT, FUTURE_SCOPE
```

Reproduction:

```
cd backend && python -m venv venv
venv/Scripts/pip install -r requirements.txt -r requirements-dev.txt
copy .env.example .env   (add GROQ / GEMINI / DEEPGRAM / ELEVENLABS keys for AI and voice)
venv/Scripts/python -m uvicorn main:app --port 8002
cd frontend && npm install && npm run dev        (http://localhost:5173)
venv/Scripts/python -m pytest                      (57 tests)
venv/Scripts/python experiments/run_all.py         (results in experiments/results/)
```

Annexure C — Evaluation Output Example

Structured evaluation JSON produced by the Evaluation agent for one answer (live run, Groq model). Scores are on a 0–10 scale; the overall score is the weighted sum.


```
{
  "technical_correctness": 2.0,
  "conceptual_understanding": 1.5,
  "answer_completeness": 1.0,
  "communication_clarity": 3.5,
  "overall_score": 2.35,
  "feedback": "The answer described LangGraph routing but did not address the FAISS trade-
offs or scaling strategy that were asked.",
  "gaps": ["No comparison with other vector stores", "No scaling plan for millions of
resumes"],
  "next_difficulty": "easy",
  "evaluation_source": "llm"
}
```